Assignment 2 Submission

Git Repository URL

https://github.com/lqc412/CS6650/tree/A2

The Git repository contains a new folder named "Server2" where you can find the complete server code, including the servlet implementation, RabbitMQ configuration, and consumer code.

Client Update:

- The second phase now uses **168 threads** instead of a dynamic number to better manage resource allocation and balance the load.

Project Components:

Server (SkierServlet):

- **Packages Used**:
  - `entity.*`: Contains classes like `LiftRide` and `ResponseMsg` used for data modeling.
  - `org.apache.commons.pool2.*`: Used for connection pooling to efficiently reuse RabbitMQ channels.
- **Classes**:
  - `SkierServlet`: This class is the main entry point for requests. It uses RabbitMQ to publish incoming skier lift ride data to the message queue (`SkierServletPostQueue`).
  - **Channel Pool**: Implements a `GenericObjectPool` to manage RabbitMQ channels, ensuring optimal reuse of connections for high throughput.

RabbitMQ Consumer (MultiThreadConsumer):

- **Packages Used**:
  - `com.rabbitmq.client.*`: Used for connecting to RabbitMQ.
  - `java.util.concurrent.*`: Manages multi-threaded operations.
- **Classes**:
  - `MultiThreadConsumer`: Listens to the message queue (`SkierServletPostQueue`) for incoming skier ride messages and processes them using multiple threads to maximize throughput. The consumer uses **200** threads to ensure messages are processed as quickly as possible. Each

consumer thread uses `channel.basicQos(30)` to limit the number of unacknowledged messages per consumer, allowing for more efficient flow control and preventing overload.

Relationships:

- The **client** sends multiple POST requests containing skier lift ride data to the **server**.
- The **server** (via `SkierServlet`) takes the incoming requests and publishes them to RabbitMQ using a pool of channels to ensure quick handling of each request.
- The **consumer** listens to the RabbitMQ queue, consumes messages, and processes them further for storage or other processing.

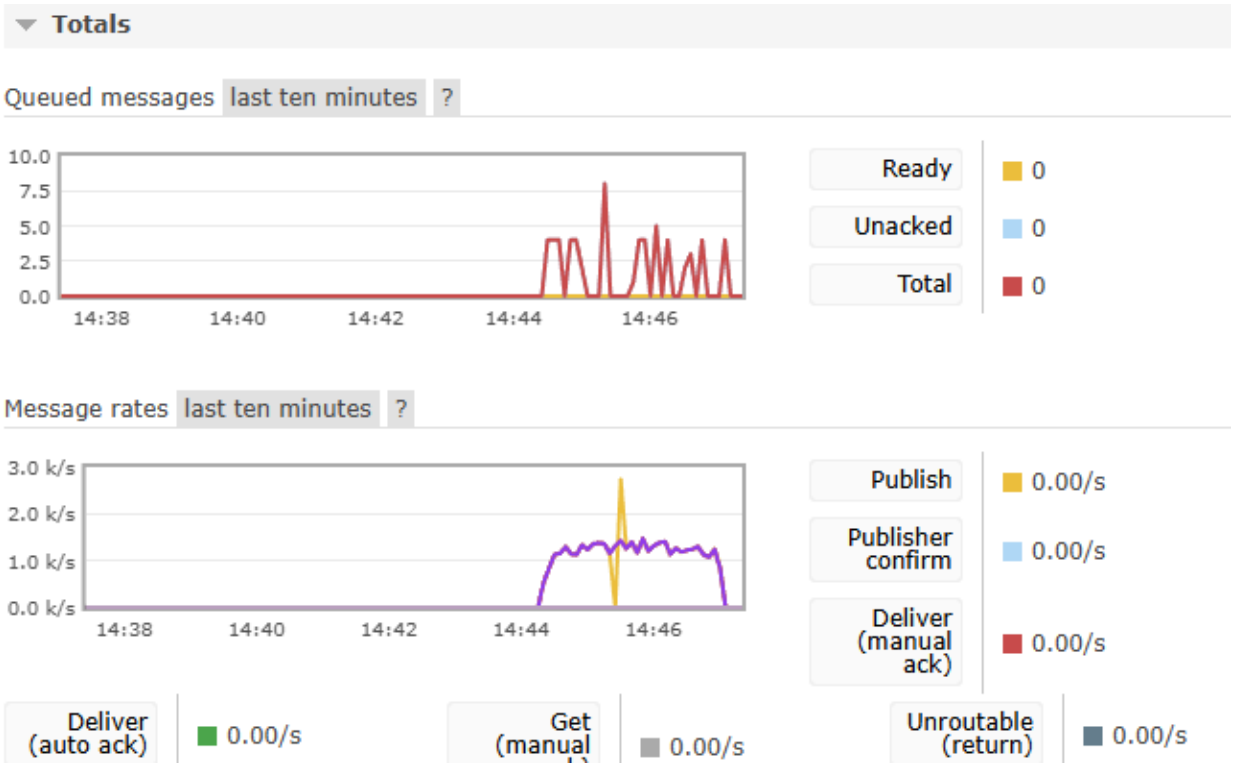
Message Flow:

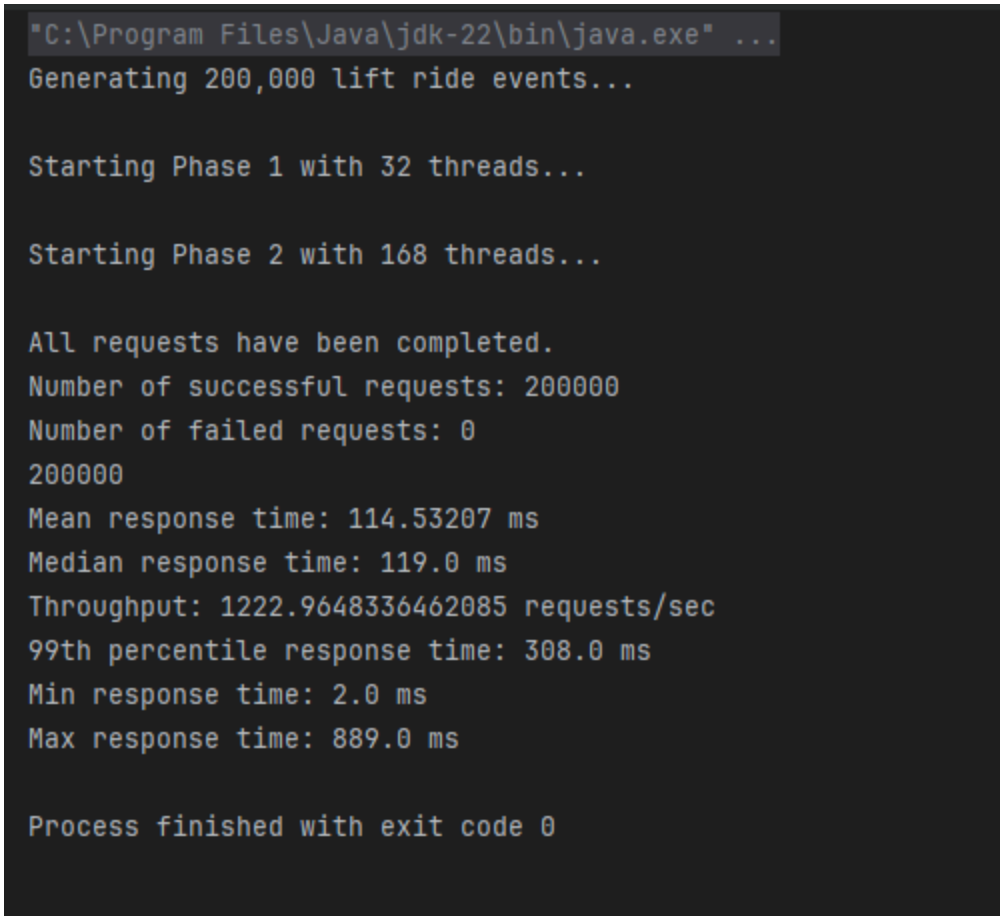
1. **Client** generates 200,000 skier lift ride events.
2. Events are sent as POST requests to **SkierServlet**.
3. **SkierServlet** publishes these requests to a RabbitMQ queue.
4. **MultiThreadConsumer** reads from the queue, acknowledges the message, and processes the skier data.

Local vs EC2 Performance:

- During local testing, the throughput is lower compared to EC2 due to CPU limitations on local hardware. Moving the entire setup to EC2, which provides more robust processing power, results in significantly better performance.

Local:


Totals
Queued messages last ten minutes ?
10.0
7.5
5.0
2.5
0.0
14:38
14:40
14:42
14:44
14:46
Ready 0
Unacked 0
Total 0
Message rates last ten minutes ?
3.0 k/s
2.0 k/s
1.0 k/s
0.0 k/s
14:38
14:40
14:42
14:44
14:46
Publish 0.00/s
Publisher confirm 0.00/s
Deliver (manual ack) 0.00/s
Deliver (auto ack) 0.00/s
Get (manual 0.00/s
Unroutable (return) 0.00/s



Single instance:

Overview

Totals

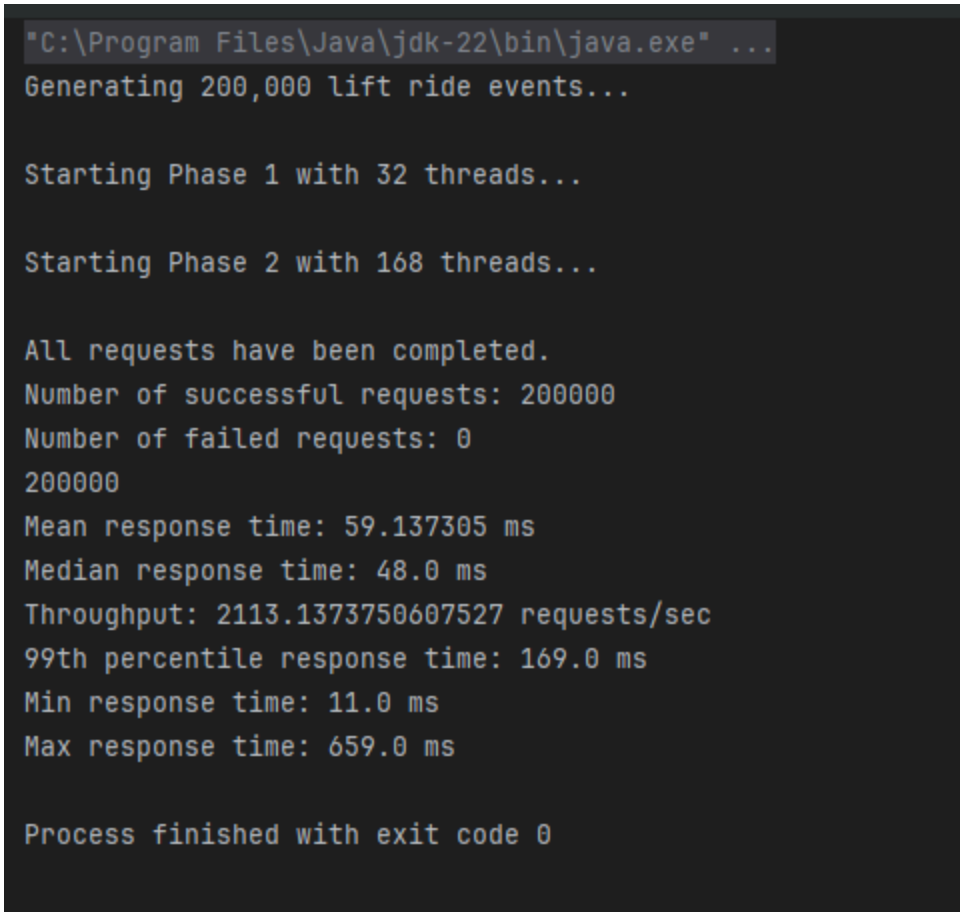
Queued messages last ten minutes ?

Ready	0
Unacked	0
Total	0

Message rates last ten minutes ?

Publish	0.00/s
Publisher confirm	0.00/s
Deliver (manual ack)	2.2/s

Global counts ?

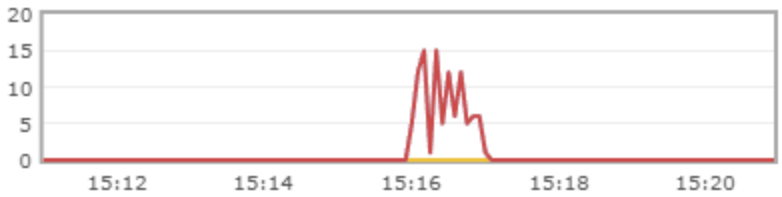


Tow Instances:

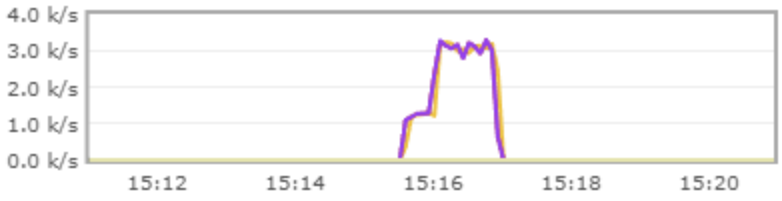
Overview

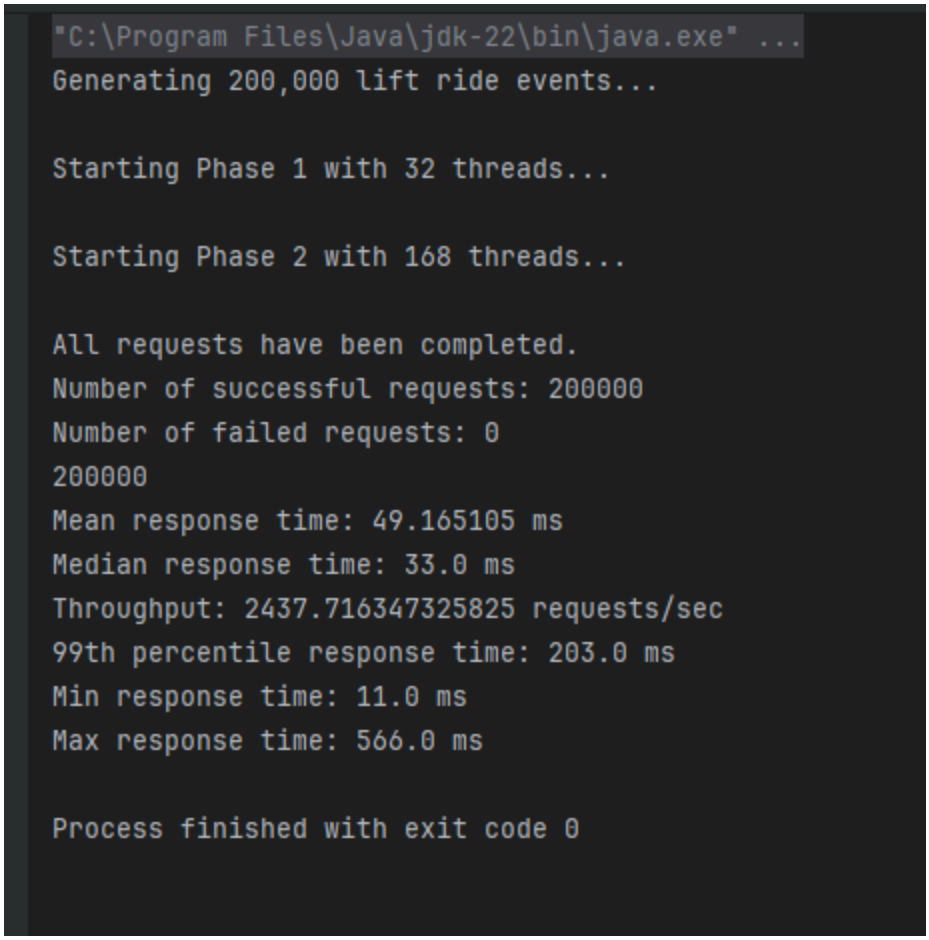
Totals

Queued messages last ten minutes ?



Message rates last ten minutes ?



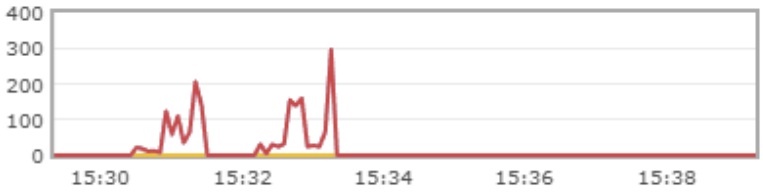


Four Instances:

Overview

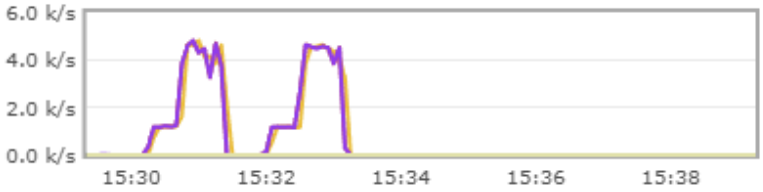
Totals

Queued messages last ten minutes ?



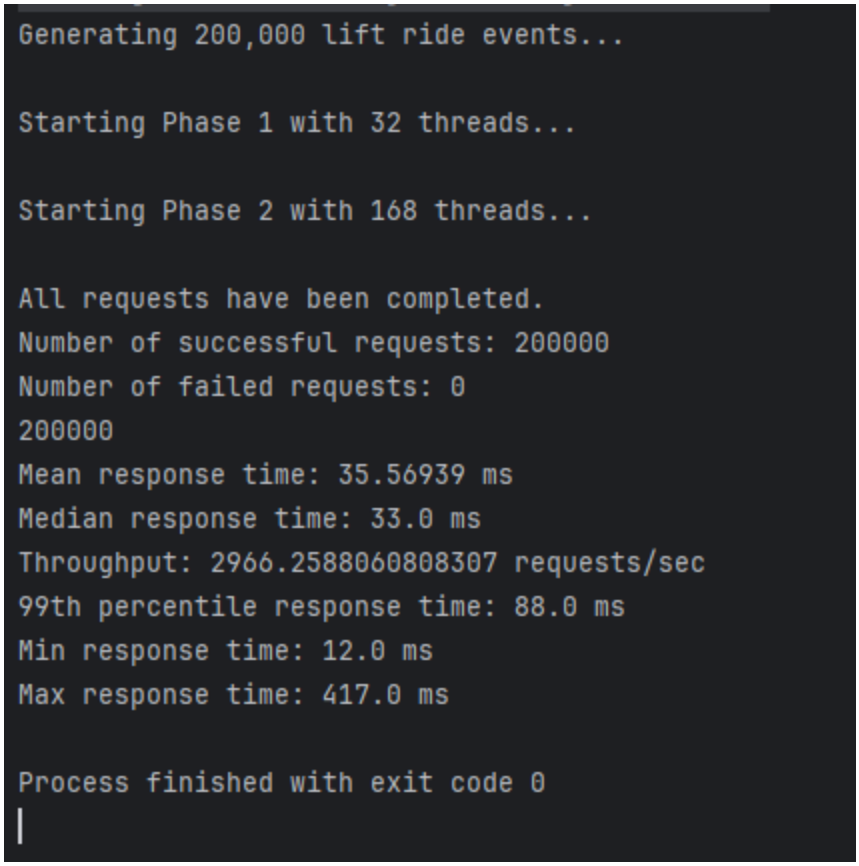
Ready	0
Unacked	0
Total	0

Message rates last ten minutes ?



Publish	0.00/s
Publisher confirm	0.00/s
Deliver (manual ack)	0.00/s

Global counts ?



Summary:
By using different instances, we can see that more instances can significantly improve throughput. Load balancing improved overall system throughput, reduced individual server load, and provided better scalability.